

第 15 讲：ContextBench：把 Context Retrieval 从最终修复率里拆出来

Coding Agent Notes (June 2026)

本讲在系列中的位置： #13 讲 context engineering，#14 讲 repo map / AST / LSP；本讲讲怎样评测 agent 是否真的找到了正确 context。

已知前提： 读者已经知道 context budget、repo intelligence、runtime evidence。

本讲新增： gold context、context recall / precision / F1、explored vs used gap、retrieval efficiency、过程级 context 评测。

读完后应该能回答： 为什么 Pass@1 不能说明 agent 会找上下文；为什么高 recall 可能仍然差；为什么检索到 gold context 不等于最终 patch 用到了它。

1 核心图景

传统 coding-agent benchmark 主要看最终结果：patch 是否通过 tests，issue 是否解决，Pass@1 是多少。这个视角很重要，但它隐藏了一个关键过程：agent 到底有没有找到解决问题所需的代码上下文？



ContextBench 的贡献，是在 issue-resolution 任务中加入 human-verified gold context。这样评测不只看最终 patch，还能看 agent trajectory 中探索过哪些文件、blocks、lines，它们和 gold context 的 overlap 如何，检索过程是过度召回还是精确命中，最终 patch 是否真正利用了已经探索到的关键上下文。

Insight. ContextBench 把“修好了没有”拆成两个问题：agent 有没有找到该看的代码？找到之后有没有用起来？这让 context retrieval 从最终成功率里被单独看见。

2 为什么最终成功率不够

一个 agent 可能在没有完整 gold context 的情况下修对任务，因为 patch 很小、测试很强、错误局部明显。另一个 agent 可能读到了关键文件，但最后 patch 没有利用这些信息。只看 Pass@1，这两类行为会被混在一起。

情况	Pass@1 视角	ContextBench 视角
没找全但修对	成功	可能是局部或侥幸成功
找全但没修对	失败	retrieval 不是瓶颈
找很多但噪声大	可能成功或失败	recall 高、precision 低
找到了但丢掉	轨迹不可见	explored-used gap

这正是过程评测的价值。它不是替代最终成功率，而是解释最终成功率从哪里来。对于系统设计者来说，如果失败来自 context retrieval，就要改检索、repo map、AST/LSP、trajectory compaction；如果失败来自 patch generation，就要改编辑格式、验证、repair strategy。

3 Gold Context

ContextBench 的 gold context 是人工验证的紧凑充分上下文。论文数据集包含 1,136 个 issue-resolution tasks, 来自 66 个 repositories, 覆盖 8 种编程语言。Gold contexts 覆盖 4,548 files、23,116 blocks/classes/functions、522,115 lines。

项目	数量
Issue tasks	1,136
Repositories	66
Programming languages	8
Gold-context files	4,548
Gold blocks / classes / functions	23,116
Gold-context lines	522,115

Gold context 的含义不是“唯一正确答案”。一个 issue 可能有多个合理 patch, 多个 patch 对应的上下文集合不完全一样。ContextBench 论文也做了 robustness analysis, 报告 semantically equivalent patches 下 gold contexts 的一致性 Jaccard similarity 为 0.9518。这个数字说明 gold context 在论文设置里相当稳定, 但仍然要把它理解成评测 reference, 而不是所有可能解的全集。

4 ContextBench 的基本对象

为了把 context retrieval 单独拿出来评测, 先要把一个 issue-resolution trajectory 拆成几个对象。设任务为 T , 人工验证出的必要上下文为 C_G , agent 在执行过程中第 t 步看到的上下文为 C_t , 到第 t 步累计看过的上下文为 $A_t = \cup_{i \leq t} C_i$ 。最后用于 patch 或 final reasoning 的上下文可以记作 $\tilde{A}$ 。

这个记法背后的直觉很简单: 最终 patch 只是终点, ContextBench 关心的是 agent 在到达终点之前经过了哪些代码证据。一个高质量 agent 不只是“最后生成了一个 patch”, 还应该能在有限预算内找到相关文件、聚焦到函数/类级片段、保留关键证据, 并把这些证据转化成修改。

对象	含义	读法
T	issue-resolution task	需要解决的问题实例
C_G	gold context	人工验证的紧凑充分上下文
C_t	step context	第 t 步工具或模型看到的内容
A_t	explored context	到第 t 步累计探索过的内容
$\tilde{A}$	utilized / patch context	最终 patch 实际依赖的内容

这里最容易误解的是 gold context。它不是“把整个仓库里所有可能相关的东西都列出来”, 而是面向解决当前 issue 的 compact sufficient reference。Compact 的意思是尽量少放噪声; sufficient 的意思是足以解释为什么 patch 应该这样改。

5 数据构造: 从 issue 到 gold context

ContextBench 的数据构造不是简单把现有 issue benchmark 拿来跑模型。它先选择真实 issue-resolution 任务, 再结合参考 patch、测试结果和人工标注, 得到每个任务对应的 gold context。论文强调 human-in-the-loop 标注, 因为“相关上下文”不是纯字符串匹配可以可靠决定的。

阶段	做什么	为什么需要
Task selection	筛选 issue-resolution instances	避免重复、低质量或不可验证任务
Patch tracing	从 gold patch 追踪依赖	找到修改背后的代码证据
AST alignment	用 structural coordinates 对齐文件、block、line	让不同 agent 轨迹可比较
Human review	人工确认 compact / sufficient context	降低机械标注误差
Robustness check	比较等价 patch 的 context 一致性	检查多解问题是否破坏 reference

这个流程有一个工程上很重要的含义：ContextBench 评测的不是“模型会不会全文搜索”，而是“模型在真实代码结构里能不能找到解决任务所需的结构化证据”。这也是为什么它同时报告 file、block、line 三个粒度。仓库级任务的难点常常不在某一行代码本身，而在“知道应该看哪个模块、哪个类、哪个函数、哪个调用关系”。

Insight. Gold context 是给评测用的参考坐标系。没有这个坐标系，agent 轨迹只能被最终 patch 成败粗略评价；有了它，才能把“找不到”“找太多”“找到了但没用”拆开。

6 轻量 Gold Context 标注协议

论文里的完整人工标注成本很高，但团队内部不一定一开始就复刻完整流程。更现实的做法，是先为失败任务维护一个 lightweight gold context protocol。它不追求覆盖所有任务，而是选择最能暴露系统问题的样本：失败任务、成本异常高的任务、patch 看似通过但 review 不可信的任务。

轻量标注可以按三层进行。第一层是 evidence file：哪些文件是解决 issue 必看的。第二层是 evidence block：哪些函数、类、配置段、测试用例真正解释了 bug。第三层是 evidence relation：这些片段之间有什么依赖关系，例如测试触发哪个路径，配置影响哪个分支，异常来自哪个调用链。

标注层	要记录什么	为什么有用
File	必要源码、测试、配置、文档文件	判断入口检索是否失败
Block	函数、类、方法、配置段、测试 case	判断定位粒度是否合适
Relation	caller/callee、test-to-code、config-to-behavior	判断 agent 是否理解因果链

Relation 这一层尤其重要。很多 repo issue 不是“某一行错了”，而是测试、接口、状态、配置和实现之间的关系被破坏。如果只标 file 或 line，agent 可能看起来命中了 gold context，却仍然没有理解为什么要改这个位置。把 relation 写出来，可以帮助审计者区分“看见代码”和“理解证据链”。

轻量协议还应该记录 negative evidence。也就是说，哪些 agent 打开的文件看起来相关但实际是噪声。这个信息对 precision 诊断很有价值。比如 agent 反复打开日志、README 或相邻模块，可能说明 issue parser 被关键词误导；agent 打开了太多测试但没打开实现，可能说明它停留在症状层，没有追到原因层。

内部实践：

每周选择 10-20 个代表性失败任务做轻量 gold-context 审计，比盲目增加几百个最终通过率样本更能暴露 agent 系统的结构性问题。

7 基本指标

Context recall 衡量 agent 覆盖了多少 gold context。Precision 衡量 agent 检索到的内容里有多少是 gold-relevant。F1 是二者的 harmonic mean。论文在 file、block、line 粒度上计算这些指标，因为不同粒度回答不同问题。

指标	形式	解释
Recall	$ A \cap C_G / C_G $	gold context 里有多少被覆盖
Precision	$ A \cap C_G / A $	agent 看到的内容里有多少相关
F1	$2PR/(P + R)$	recall 与 precision 的折中

这里的 A 可以是最终累计探索上下文，也可以是某个阶段的上下文集合。评测时必须说明粒度和阶段，否则同一个数字可能表达完全不同的行为。比如 file-level recall 高，可能只是打开了一个巨大的文件；line-level precision 高，可能说明 agent 非常聚焦，但也可能说明它过早收窄，漏掉调用链上的必要证据。

粒度	衡量什么	注意事项
File	是否找到相关文件	粗，但稳定
Block	是否找到函数/类级上下文	更接近修复单元
Line	是否命中精确代码行	精细，但对等价实现敏感
Process	何时找到、是否冗余、是否使用	能解释轨迹质量

这些指标让我们能区分三种失败：没有找到 gold context；找到了但带来大量噪声；找到了但后续没有利用。对 agent 训练或系统调参来说，这三类失败需要完全不同的修复手段。只看最终 Pass@1，这些失败会被压成一个“没修好”。

8 粒度为什么重要

File、block、line 三个粒度并不是重复报告，而是在回答三个不同层次的问题。File-level 指标问 agent 是否知道大概去哪看；block-level 指标问 agent 是否定位到真正的类、函数、方法；line-level 指标问 agent 是否命中具体修改附近的证据。

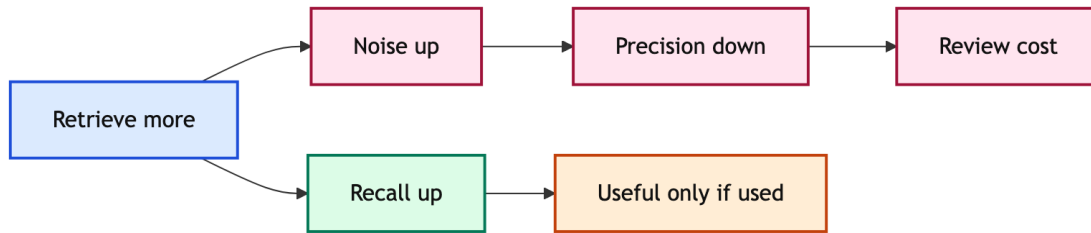
粒度	常见假阳性	常见假阴性
File	打开一个大文件导致 recall 虚高	相关逻辑分散在多个小文件
Block	命中相邻函数但没命中真正逻辑	依赖跨函数、跨类、跨模块
Line	patch 附近行命中但原因没覆盖	等价 patch 改了不同位置

这也是为什么 repo map、AST、LSP 在 #14 之后要接上 ContextBench。一个只按文件名搜索的 agent 可能 file-level 还不错，但 block-level 很差；一个会用 AST / LSP 的 agent 可能更容易把检索单位控制在 symbol、definition、reference、caller/callee 这些结构上。ContextBench 提供的不是工具，而是判断工具是否真的改善上下文定位的度量。

读指标的基本顺序：

先看 file-level recall，判断 agent 有没有找到区域；再看 block-level F1，判断定位是否可用；最后看 line-level precision 和 explored-used gap，判断它是否真正把证据用在 patch 上。

9 Recall vs Precision



ContextBench 的一个重要发现，是被评测模型和 agents 往往倾向于提高 recall 而牺牲 precision。换句话说，它们宁愿多读很多不相关上下文，也不愿漏掉关键上下文。这在最终修复任务里有直觉：漏掉关键文件可能必败；多读一点也许还有机会。

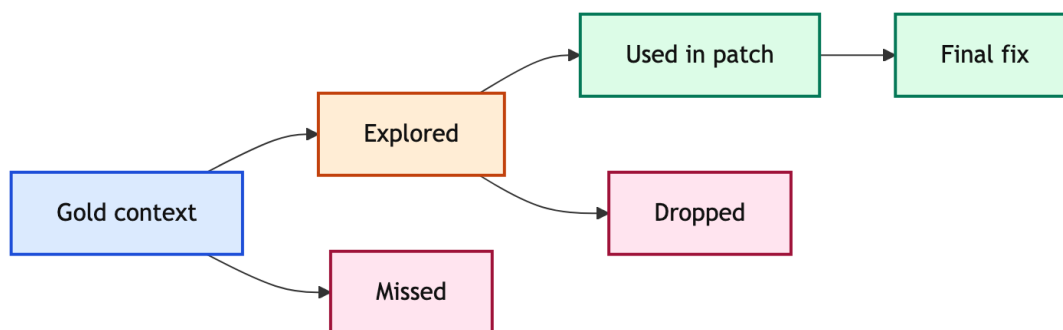
但对长期 agent 系统来说，低 precision 有真实成本。它会占用 context budget，增加工具调用，放大 review 负担，让模型在噪声里失焦。高 recall 只有在模型能选择、压缩并最终使用相关证据时才有价值。

检索风格	好处	代价
High recall	不容易漏关键文件	噪声、成本、上下文污染
High precision	context 干净、review 轻	可能漏掉必要依赖
Balanced	修复和成本更稳	需要好 granularity 和排序

如果把 context retrieval 看成一个搜索问题，recall 和 precision 的张力来自两个方向。真实软件任务的相关代码经常分散在配置、测试、接口、实现、错误处理、依赖版本之间，所以过早收窄会漏证据。另一方面，模型上下文窗口和工具预算有限，读太多文件会让后续推理变差。ContextBench 的价值在于把这个 tradeoff 量化出来，让系统设计者不再只凭最终成败猜测 retrieval policy 好不好。

现象	可能原因	调整方向
Recall 低、precision 高	搜索过早收敛	扩大入口、增加 dependency expansion
Recall 高、precision 低	盲目打开大文件或长日志	改 ranking、summarization、symbol granularity
F1 高但 Pass@1 低	找到了但不会修	改 patch generation 和 validation loop
Pass@1 高但 recall 低	任务局部或测试强	不要把它误读成检索能力强

10 Explored vs Used



另一个关键概念是 explored context 和 utilized context 的差距。Agent 可能在轨迹中打开过 gold-relevant 文件，但最终 patch 或 final reasoning 没有用它。论文把这种现象称为 retrieval 和 use 的 gap。

这对 #13 的 compaction 也很重要。长任务里，agent 会读很多文件；如果中间证据没有被保留、重排、引用或压缩到工作状态里，后面生成 patch 时就像没读过一样。换句话说，context retrieval 不是一次性动作，而是“找到、保留、选择、利用”的链路。

诊断问题：

如果 agent 失败，先问：它没找到 gold context，还是找到了但没用？前者是 retrieval 问题，后者是 context management / reasoning / patch generation 问题。

11 过程指标：早找到、少重复、能保留

最终累计 recall 仍然不够。两个 agent 都可能在结束前找到同样多的 gold context，但一个在第 3 步就找到了，另一个在第 30 步才找到；前者显然更高效，也更适合长任务。ContextBench 因此关心 process-level dynamics，包括效率、冗余和 explored-utilized gap。

过程信号	衡量什么	坏味道
AUC-Cov / efficiency	gold context 是否早被覆盖	很晚才找到关键证据
Redundancy	新检索内容是否重复旧内容	反复打开同一文件、循环阅读
Drop / Keep	观察到的 gold evidence 是否保留到最终 patch context	读过关键代码但最终没用
Cost	工具调用、token、时间成本	靠暴力搜索堆 recall

这组指标比最终 recall 更接近真实 agent 产品的体验。用户不只关心 agent 最终是否成功，也关心它是否一直绕圈、是否读了大量无关内容、是否能把前面发现的证据带到后面的编辑决策里。对于生产系统，早发现、少重复、能保留，比单纯“最后覆盖过 gold context”更重要。

12 四类失败画像

把 Pass@1、recall、precision、process dynamics 放在一起，就能画出四类常见失败画像。这些画像比“模型不够强”更有工程指导意义。

失败画像	表现	更可能的修复
入口错误	file-level recall 很低	改 issue parsing、repo map、keyword/symbol search
范围失控 记忆丢失	recall 尚可但 precision 很低 explored recall 高但 utilized keep 低	改 top-k、chunking、AST block、ranking 改 trajectory summary、evidence card、state persistence
编辑失败	context 指标好但 Pass@1 低	改 patch format、validation、repair loop

这四类失败对应不同团队分工。入口错误通常是检索和仓库理解团队的问题；范围失控是 context engineering 问题；记忆丢失是 agent runtime 和 trajectory management 问题；编辑失败则更接近 model capability、tool interface、validation harness 的问题。把它们混在一起调参，会让改动方向不稳定。

13 论文报告的主要发现

按论文表述, ContextBench 的评测显示几类现象。第一, 更复杂的 agent scaffolding 对 context retrieval 的提升有限, 作者把这称为 coding agents 的“Bitter Lesson”式结果: 复杂机制不一定带来更好的上下文检索。第二, LLMs 和 agents 普遍偏向 recall over precision。第三, balanced retrieval frequency 和 context granularity 能改善 Pass@1 与 cost efficiency。第四, explored 和 utilized context 之间存在明显缺口。

发现	如何理解
Scaffolding limited gains	复杂 agent 结构不自动改善 context finding
Recall over precision	模型倾向多看, 代价是噪声
Balanced retrieval matters	检索频率和粒度需要调节
Explored-used gap	读过不等于用过

这些结论不应该被读成“agent scaffold 没用”。更准确的读法是: 如果 scaffold 不能改善 context selection、retention 和 utilization, 它在 context retrieval 这个子问题上贡献有限。

14 怎样读 ContextBench 结果

第一, 不要把 context metrics 当作 patch correctness。高 recall 不保证修复正确; 低 recall 也可能局部修对。ContextBench 的作用是解释过程, 而不是替代 executable validation。

第二, 要按粒度读。File-level recall 高, 可能只是找到了大文件; line-level precision 低, 可能说明检索太粗。Block-level 指标通常更贴近 coding agent 的编辑单元, 但也依赖 annotation granularity。

第三, 要结合成本读。同样 recall 下, 早期找到 gold context 比很晚才找到更好; 少量工具调用找到比大量盲搜更好; 把 gold context 留在最终 patch reasoning 中比只是轨迹中路过更好。

不要这样读	更好的读法
“某 agent Pass@1 高, 所以 context retrieval 强。”	先看 context recall / precision, 再看是否存在局部任务或强测试掩盖。
“Recall 高就是好。”	同时看 precision、cost、冗余和 final utilization。
“复杂 scaffold 没用。”	只说明当前 scaffold 没显著改善 retrieval; 也许它改善了安全、可恢复性或人机协作。
“Gold context 是唯一答案。”	Gold context 是稳定 reference, 但多解任务仍要谨慎解释。

15 和训练、奖励、评测的关系

ContextBench 还可以被看成 reward design 的原型。传统 RL 或 SFT 数据常常只有最终结果: patch pass 或 fail。ContextBench 提供中间过程信号: 是否找到 gold context、何时找到、是否过度检索、是否保留。这样的信号能更细地奖励 agent 的探索过程。

但这不意味着应该直接把 gold-context overlap 作为唯一奖励。过度优化 recall 会鼓励 agent 打开大量文件; 过度优化 precision 会鼓励 agent 只看很少内容, 漏掉长距离依赖。更合理的奖励应该把 retrieval quality、cost、validation result 和 final correctness 放在一起。

训练信号	能奖励什么	需要防什么
Context recall	覆盖必要证据	暴力读取全部仓库
Precision / F1	聚焦相关上下文	过早停止探索
Early coverage	快速定位	牺牲深层依赖
Keep / Drop	保留关键证据	机械引用而不理解
Executable success	真正修复问题	忽略过程可解释性

这也解释了为什么 ContextBench 对 agent 产品有价值。它不是只给一个 leaderboard，而是能告诉开发者哪个模块在拖后腿：retriever、ranker、memory、editor、validator，还是预算策略。

16 和 #13、#14 的连接

#13 讲的是 context engineering：怎样在有限窗口里组织信息。ContextBench 给它补上评测方法。一个 context compression 策略如果让 utilized keep 下降，说明它把关键证据压丢了；如果让 precision 上升但 recall 大幅下降，说明它过度清洗了上下文。

#14 讲的是 repo map / AST / LSP：怎样把仓库变成可导航结构。ContextBench 给它补上目标函数。Repo map 不应该只追求“看起来覆盖很多文件”，而应该提高 block-level F1、减少重复探索、让 agent 更早找到 gold context。

前序主题	ContextBench 给出的验收	反例
Context compaction	gold evidence 被保留到 final state	summary 很短但漏掉关键调用关系
Repo map	相关 symbol 更早被发现	打开很多文件但定位不到函数
AST/LSP tools	block-level precision 提升	工具返回过大范围上下文
Validation harness	context 好坏能和 patch 成败分开看	只知道 fail，不知道为什么 fail

17 工程 Checklist

如果要把 ContextBench 的思想迁移到自己的 coding-agent 系统，不一定必须完整复刻 benchmark。更实用的做法，是在内部任务上维护一个轻量 context audit。

检查项	具体问题
Gold evidence	每个失败任务是否能标出必要文件、函数、测试和配置？
Trajectory capture	每次工具调用看到的文件、range、symbol 是否被记录？
Granularity	检索单位是文件、chunk、函数、类，还是 line range？
Ranking	agent 为什么先看这些文件？排序信号是什么？
Retention	关键证据是否进入后续 summary、plan、patch rationale？
Budget	高 recall 是否靠大量重复读取换来？
Failure split	失败时能否区分 retrieval、reasoning、editing、validation？

这个 checklist 的目的不是制造更多报表，而是避免团队在错误层面优化。例如，patch editor 失败时继续加搜索工具没有意义；retrieval 失败时换更复杂的 diff format 也不会解决根因。

18 怎样审计一次失败轨迹

把 ContextBench 用在日常系统调试里，可以从一次失败轨迹开始。假设 agent 最终 patch 没有通过测试。不要立刻问“模型为什么没写对”，而是按证据链往前追。

第一步，列出 gold evidence。至少要标出 issue 描述指向的测试、失败栈、相关配置、被修改模块和关键调用链。哪怕没有完整人工 gold context，也可以先做一个 lightweight reference set。

第二步，回放 agent trajectory。记录它打开过哪些文件、查看过哪些函数、运行过哪些测试、保留了哪些中间结论。这里最重要的是区分“曾经看到”和“最终使用”。很多 agent 的失败不是没看见，而是看见后没有把证据带到 patch decision 中。

第三步，把失败归因到最小层。若它从未打开关键文件，问题在 entry retrieval；若打开了大文件但没定位函数，问题在 granularity；若看过关键函数但 summary 没保留，问题在 memory；若证据保留了但 patch 错，问题才更可能在 editing 或 reasoning。

审计步骤	需要的记录	产出
Gold reference	必要文件、函数、测试、配置	一个可比较的 context set
Trajectory replay	tool calls、opened ranges、summaries	explored context 时间线
Utilization check	final plan、patch rationale、diff	evidence 是否进入最终决策
Root-cause split	recall / precision / keep / pass	下一轮该改哪个模块

Insight. 失败轨迹审计的目标，是把“agent 没修好”改写成一个可行动判断：它没找到、找太多、找到了但丢掉，还是找到了也用上了但不会改。

19 设计启发

对 agent 系统来说，ContextBench 给出三条直接启发。第一，retrieval 应该有 granularity control: file、function、class、line 不能混用一套策略。第二，trajectory 需要保留 gold-relevant evidence，而不是读完就丢。第三，检索模块应该被单独评测，避免把 patch generator 的强弱误认为 context retriever 的强弱。

系统部件	ContextBench 诊断	可能改进
Search / repo map	recall/precision 低	改 ranking 和 granularity
Trajectory memory	explored-used gap 大	保留 evidence cards
Patch generation	retrieval 好但修复差	改 editing/validation
Budget policy	recall 高 precision 低	限制噪声和冗余

因此，ContextBench 对系统设计的真正启发是“分层验收”。一个 coding agent 至少要同时通过三层：能找到上下文，能管理上下文，能利用上下文修复问题。任何一层失败，最终 Pass@1 都会受影响；但只有把中间过程拆开，才知道下一轮应该改哪里。

20 从 Leaderboard 到诊断面板

如果只把 ContextBench 做成 leaderboard，它的价值会被压扁。更好的使用方式，是把它当成诊断面板：每次系统改动后，不只报告 Pass@1，还报告 context retrieval 的过程曲线。这样能避免“分数涨了但原因不明”。

一个有用的诊断面板至少应该有四行。第一行是最终效果：Pass@1、resolved rate、平均验证次数。第二行是上下文质量：file/block/line recall、precision、F1。第三行是过程效率：平均检索轮数、早期 gold coverage、重复读取比例、平均成本。第四行是利用质量：observed gold evidence 有多少进入最终 patch reasoning。

面板区域	指标	回答的问题
Final success	Pass@1、resolved rate	最后修好了多少
Retrieval quality	recall、precision、F1	有没有找到正确上下文
Process cost	rounds、AUC-Cov、redundancy、tokens	找得早不早、贵不贵、绕不绕
Utilization	keep/drop、patch evidence	读到的证据是否真正用上

这个面板还可以帮助比较系统改动。例如，引入 AST search 后，如果 block-level precision 上升但 Pass@1 没变，说明检索已经更干净，下一步可能要看 patch editor 或 validator。如果加入 long-context summarizer 后 Pass@1 上升但 keep/drop 变差，就要警惕模型可能靠更强生成能力掩盖了证据保留问题。

换句话说，ContextBench 让系统迭代从“盯一个最终分数”变成“看一组互相约束的过程信号”。这和 DDIA 式系统分析很接近：不要只问吞吐或延迟一个数字，而要同时看正确性、成本、可恢复性、可解释性和维护性。

21 如何报告一次 ContextBench 风格实验

在真实团队里，ContextBench 最有价值的用法不是单独发布一个总分，而是把每次 agent 改动写成可诊断的实验报告。一个好的报告应该让读者知道三件事：系统改了什么，context 行为怎样变了，最终修复率的变化能不能被这些中间信号解释。

最小报告单元可以按四块组织。第一块是实验设置：模型版本、harness 版本、工具集合、预算、任务子集、是否允许 internet 或额外文档。第二块是最终指标：Pass@1、平均尝试次数、验证成本。第三块是 context 指标：file/block/line 的 recall、precision、F1，以及早期覆盖和冗余。第四块是失败分析：列出若干典型失败轨迹，说明失败属于 retrieval、retention、editing 还是 validation。

报告块	必须记录	避免误读
Setup	model、harness、tools、budget、task split	不同设置的分数不能直接比较
Final outcome	Pass@1、validation pass、cost	最终成功率不解释原因
Context behavior	recall、precision、F1、AUC-Cov、redundancy	高 recall 可能只是读太多
Failure cases	失败轨迹和 gold evidence 对照	不要用一个失败故事代表全部任务

这种报告方式会改变团队讨论方式。以前看到 Pass@1 下降，大家可能只会说模型变差了；现在可以进一步问：是不是新 prompt 让 agent 更少打开测试？是不是新 repo map 提高了 file recall 但降低了 block precision？是不是 summarizer 把关键证据压掉了？是不是 validator 更严格，所以最终成功率下降但 context retrieval 其实变好了？

报告原则：

不要只报告“新系统赢了旧系统”。要报告它赢在哪里、输在哪里、代价是什么，以及这个变化是否来自 context retrieval 本身。

对长期知识库来说，这类报告还有一个额外好处：它能沉淀 failure taxonomy。几个月后再看历史实验，团队可以知道哪些失败已经被 repo map 改掉，哪些失败属于工具接口，哪些失败始终是 patch generation 的硬问题。这样 ContextBench 不只是一次 benchmark，而是持续改进 agent 系统的观察语言。

22 局限与开放问题

Gold context 需要人工标注，成本高。Gold context 也依赖参考 patch；即使论文报告高一致性，真实项目里仍可能有多个架构层面的解决方案。ContextBench 评估的是 context retrieval，不直接评估长期 memory reuse、跨任务 experience transfer，这些会留给 #16 SWE-ContextBench。

此外，context metrics 本身也可能被 gaming。一个系统可以通过读取大量文件提高 recall，但这会降低 precision 和效率。因此报告结果时必须同时看 recall、precision、F1、efficiency、redundancy 和 final task success。

另一个局限是，ContextBench 主要刻画单个 issue-resolution trajectory 内部的 context retrieval。真实 long-horizon agent 还会遇到跨 session、跨任务、跨项目的经验复用：以前的调试经验是否能迁移到

新仓库? 失败轨迹是否能变成技能? 历史 patch 是否能帮助新 issue? 这些不是 ContextBench 的主要范围, 也正是下一讲 #16 SWE-ContextBench 会更集中讨论的方向。

23 最终 Takeaway

ContextBench 的核心意义, 是把 coding agent 的“找上下文”能力从最终修复率里拆出来。它让我们看见: agent 是否找到 gold context, 是否过度检索, 是否把读到的证据用在最终 patch 中。

对后续 notes 来说, ContextBench 提供了一个重要 vocabulary: context retrieval 不只是 prompt 长短问题, 而是一个可标注、可度量、可诊断的过程。

关键词	在本讲中的含义
Gold context	解决 issue 所需的紧凑充分代码证据, 不是整个仓库摘要。
Explored context	agent 轨迹中曾经观察过的上下文, 代表“看见过”。
Utilized context	最终 patch 或 reasoning 真正保留并使用的证据, 代表“用上了”。
Retrieval efficiency	关键上下文是否被早发现、低成本发现、少重复发现。
Context audit	对一次失败轨迹做过程归因, 区分找不到、找太多、丢证据和不会改。

参考资料

- [1] [ContextBench](#). benchmark construction, gold contexts, retrieval metrics, and process-level findings
- [2] [ContextBench Project Page](#). project page, data/code pointer, and benchmark assets